

Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчёт к практическому заданию №2
по дисциплине «Системное программное обеспечение»

Студент: Горинов Даниил Андреевич
Группа: Р4116

Санкт-Петербург
2025/2026 учебный год

1 Цели

Построить граф потока управления (CFG) по деревьям разбора набора файлов, связать узлы CFG с деревьями операций и сформировать графическое представление результатов анализа.

2 Задачи

Порядок выполнения:

1. Описать структуры данных, необходимые для представления информации о наборе файлов, наборе подпрограмм и графе потока управления, где:
 - (а) Для каждой подпрограммы: имя и информация о сигнатуре, граф потока управления, имя исходного файла с текстом подпрограммы.
 - (б) Для каждого узла в графе потока управления, представляющего собой базовый блок алгоритма подпрограммы: целевые узлы для безусловного и условного перехода (по мере необходимости), дерево операций, ассоциированных с данным местом в алгоритме, представленном в исходном тексте подпрограммы
2. Реализовать модуль, формирующий граф потока управления на основе синтаксической структуры текста подпрограмм для входных файлов
 - (а) Программный интерфейс модуля принимает на вход коллекцию, описывающую набор анализируемых файлов, для каждого файла — имя и соответствующее дерево разбора в виде структуры данных, являющейся результатом работы модуля, созданного по заданию 1.
 - (б) Результатом работы модуля является структура данных, разработанная в п. 1, содержащая информацию о проанализированных подпрограммах и коллекция с информацией об ошибках
 - (в) Посредством обхода дерева разбора подпрограммы сформировать для неё граф потока управления, порождая его узлы и формируя между ними дуги в зависимости от синтаксической конструкции: выражение, ветвление, цикл, прерывание цикла, выход из подпрограммы — для всех синтаксических конструкций по варианту
 - (г) С каждым узлом графа потока управления связать дерево операций, в котором каждая операция в составе текста программы представлена как совокупность вида операции и соответствующих операндов (см. задание 1, пп. 2.d–g)
 - (е) При возникновении логической ошибки в синтаксической структуре при обходе дерева разбора сохранить в коллекции информацию об ошибке и её положении в исходном тексте
3. Реализовать тестовую программу для демонстрации работоспособности созданного модуля
 - (а) Через аргументы командной строки программа должна принимать набор имён входных файлов, имя выходной директории

- (b) Использовать модуль, разработанный в задании 1, для синтаксического анализа каждого входного файла и формирования набора деревьев разбора
 - (c) Использовать модуль, разработанный в п. 2, для формирования графов потока управления каждой подпрограммы
 - (d) Для каждой обнаруженной подпрограммы вывести представление CFG в отдельный файл с именем “sourceName.functionName.ext” в выходной директории; по умолчанию размещать выходные файлы рядом с входным
 - (e) Для деревьев операций в графах потока управления всей совокупности подпрограмм сформировать граф вызовов и вывести его в дополнительный файл; по умолчанию размещать рядом с файлом, содержащим подпрограмму main
 - (f) Сообщения об ошибке должны выводиться тестовой программой в стандартный поток вывода ошибок
4. Результаты тестирования представить в виде отчета, в который включить:
- (a) В части 3 привести описание разработанных структур данных
 - (b) В части 4 описать программный интерфейс и особенности реализации разработанного модуля
 - (c) В части 5 привести примеры исходных анализируемых текстов для всех синтаксических конструкций разбираемого языка и соответствующие результаты разбора

3 Описание работы

Внешнее описание программы и интерфейс

CLI-утилита принимает список входных файлов и опциональный каталог вывода:

```
./app [-o output_dir] <input_files...>
```

По умолчанию результаты сохраняются рядом с входными файлами. Ошибки анализа выводятся в `stderr`.

Структуры данных

- `CFG_InputFile`: имя файла и AST (результат задания 1).
- `CFG_Block`: идентификатор блока, переходы `next/true_next/false_next`, метка и дерево операций `IRNode`.
- `CFG_Graph`: список блоков и идентификаторы входа/выхода.
- `CFG_Subprogram`: имя, сигнатура, имя файла и CFG.
- `CFG_Analysis`: коллекция подпрограмм и ошибок `CFG_Error`.
- `IRNode`: операции `CONST`, `LOAD`, `STORE`, `UNARY`, `BINARY`, `CALL`, `INDEX`, `RANGE`.

Именованние выходных файлов

Для каждой функции формируется DGML по шаблону:

```
<output_dir>/<source_name>.<function_name>.dgml
```

Дополнительно строятся:

```
fun.calls.graph.dgml  
cfg.dgml
```

Соответствие требованиям задания

Ключевой интерфейс модуля:

```
CFG_Analysis cfg_analyze_files(const CFG_InputFile* files, int file_count);
```

Функция принимает набор входных файлов с AST и возвращает структуру анализа с подпрограммами и коллекцией ошибок. Для каждой подпрограммы строится CFG, узлы которого связываются с деревьями операций.

4 Аспекты реализации

Построение CFG

Построение выполняется обходом AST подпрограммы. Для композиции фрагментов используется структура «вход + список выходов». Выходы соединяются при последующей сборке графа, что позволяет корректно обрабатывать вложенные конструкции. Примеры:

- **if/else**: создаётся условный блок, формируются две ветви и общий узел слияния.
- **while/until**: создаётся блок условия, тело цикла и обратная дуга в условие; выходы тела соединяются с узлом выхода.
- **repeat**: формируется блок тела, затем блок условия и дуга возврата в начало.
- **break**: выход из цикла оформляется переходом в заранее известный целевой блок.

Дерево операций (IR)

IR строится из AST выражений. Бинарные и унарные операции сохраняют тип операции (например, ADD и LT). Загрузка и запись переменных реализуются узлами LOAD и STORE. Индексация массива соответствует узлам LOAD/STORE с дочерними узлами INDEX и RANGE. Некорректные цели присваивания фиксируются как логические ошибки.

Сбор графа вызовов

Для каждой подпрограммы выполняется обход деревьев операций, при обнаружении узла CALL формируется дуга между функциями. Это позволяет получить глобальный граф вызовов в отдельном DGML-файле.

5 Результаты

- Реализованы структуры и алгоритмы построения CFG и IR.
- Выполнены тесты на входах `tests/input.txt` и `tests/control_flow_big.txt`.
- Сформированы графы:

```
results/ast.input.txt.dgml
results/input.txt.<function>.dgml
results/ast.control_flow_big.txt.dgml
results/control_flow_big.txt.<function>.dgml
results/fun.calls.graph.dgml
results/cfg.dgml
```

6 Выводы

Модуль CFG-анализа корректно строит граф потока управления и связывает его с деревьями операций, что соответствует требованиям задания 2. Полученные графические представления позволяют визуально проверить структуру алгоритмов и подготовить вход для генерации линейного кода на следующем этапе.